॥श्री गणेशाय नमः॥

# Optimization

## Single-Variable Differentiation

Consider a function y = f(x)

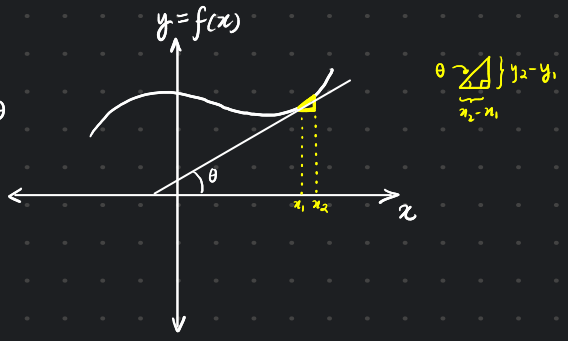
$y' = \frac{dy}{dx}$ = rate of change of $y$ w.r.t. $x$.

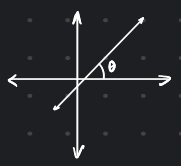
$$\frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x}$$

As $\Delta x \to 0$, we get the tangent.

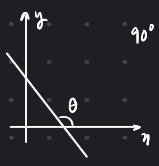
$\left[\frac{dy}{dx}\right]_{x=x_1}$ = slope of the tangent to $f(x)$ at $x = x_1$

$\frac{y_2-y_1}{x_2-x_1} = \frac{\Delta y}{\Delta x} = \tan\theta$

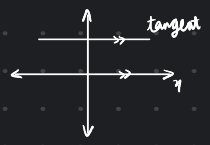



$0 < \theta < 90° \rightarrow \tan\theta$ is +ve.

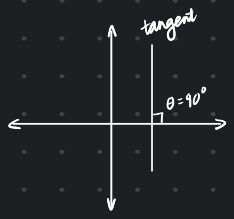



$90° < \theta < 180° \rightarrow \tan\theta$ = -ve




$\tan 0° = 0$




$\tan 90° = 1$

### Basic rules

$$\frac{d}{dx}(x^n) = n x^{n-1}$$

$$\frac{d}{dx}(f(x) + g(x)) = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

$$\frac{d}{dx}(c) = 0$$

Chain Rule:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

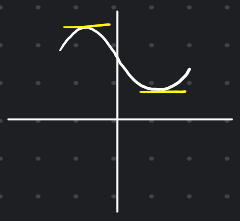
e.g. $\frac{d}{dx}(a-bx)^2 = 2(a-bx)(-b)$

$$\frac{d}{dx}\, c \cdot f(x) = c \cdot \frac{d}{dx} f(x)$$

$$\frac{d}{dx} \log x = \frac{1}{x}$$

$$\frac{d}{dx}(e^x) = e^x$$

# Maxima and Minima



At minima and maxima,
slope = 0

e.g. $f(x) = x^2 - 3x + 2$

$f'(x) = 0$ ← slope = 0 for minima

$\frac{df(x)}{dx} = 2x - 3 = 0$

$x = 3/2 \qquad f(3/2) = -\frac{1}{4}$

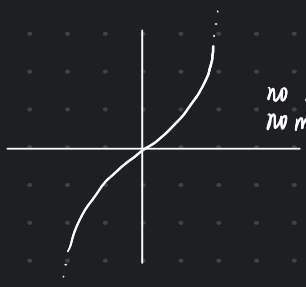
at $x = 1.5 \rightarrow f(x) = -0.25$

at $x = 1 \rightarrow f(x) = 0$

$\therefore f(1.5) \neq$ maxima.

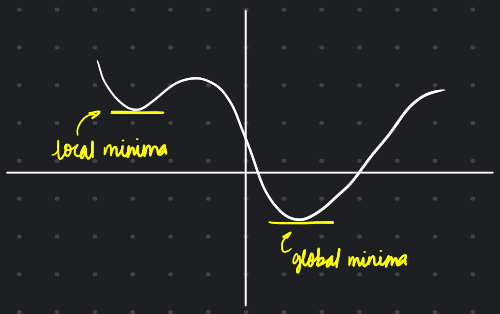
$x$ is the minima.

A func$^n$ may not have both maxima and minima.



no maxima,
no minima.

---




A func$^n$ can have slope = 0 at multiple points.

---

$f(x) = \log(1 + \exp(ax))$

$f'(x) = \frac{a \exp ax}{1 + \exp(ax)} = 0$

} solving $\frac{d}{dx} f(x)$ to get minima/maxima of $f(x)$ is not always trivial.

solution : Gradient Descent.

# Vector differentiation: Grad

say $f(x)$ : $y = a^T x$

$x = \langle x_1, x_2, \cdots, x_d \rangle$

$a = \langle a_1, a_2, \cdots, a_n \rangle$ → constants

$\frac{df}{dx} = \nabla_x f$ ← grad / del

↳ vector

$$\nabla_x f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

---

$$f(x) = a^T x = \sum_{i=1}^{d} a_i x_i$$

$$\nabla_x f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_d \end{bmatrix} = \vec{a}$$

$$\boxed{\nabla_x a^T x = a}$$

---

$$\mathcal{L}(w) = \sum_{i=1}^{n} \log\left(1 + \exp(-y_i w^T x_i)\right) + \lambda w^T w$$

→ here $\langle x_i, y_i \rangle$ are constants, since we get those from $D_{train}$.

consider this.
It'd be $\lambda w^2$ if $w$ were a scalar.
$\frac{d}{dw} \lambda w^2$ would be $2\lambda w$

$$\nabla_w L = \frac{(-y_i x_i)(e^{-y_i w^T x_i})}{1 + e^{-y_i w^T x}} + 2\lambda w$$

For minima / maxima, $\nabla_w L = 0$.

Again, this is not trivial.

# Gradient Descent Algorithm

It is an iterative algorithm

- Guess a value of x  $x_0$ ← first guess for $x^*$  We want to find $\operatorname{argmin}_x f(x)$

  ↳ iteratively apply → $x_k \sim x^*$

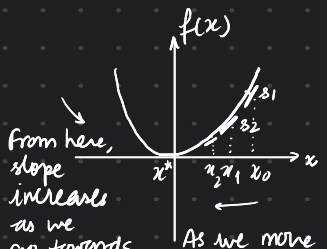
---

Geometric intuition:



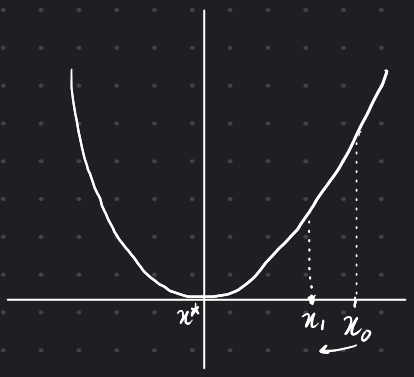
← 1. choose a point. slope at that point is +ve.

Around minima,
on one side, slope is +ve.
other side, slope is −ve.
exactly at minima, slope = 0.
so the slope changes sign at minima.




From here, slope increases as we go towards minima.

As we move towards $x^*$, slope reduces.




Now we find a $x_1$ s.t. $x_1$ is closer to $x_0$.

$$x_1 = x_0 - r\left[\frac{df}{dx}\right]_{x_0}$$

(r: step-size (constant))

say $r = 1$

$$x_1 = x_0 - \underbrace{\left[\frac{df}{dx}\right]_{x_0}}_{\text{slope (+ve)}} = x_0 - \text{some +ve value}$$

= some value bet$^n$ minima and $x_0$

So we go closer to $x^*$.

If $x_0$ was on the −ve side,




$$x_1 = x_0 - \underbrace{\left[\frac{df}{dx}\right]_{x_0}}_{\text{-ve value}}$$

$= x_0 +$ some +ve value

= closer to $x^*$.

Illy, $x_2 = x_1 - r\left[\frac{df}{dx}\right]_{x_1}$

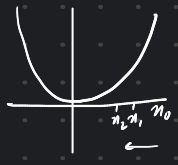
iteratively: $x_{i+1} = x_i - r\left[\frac{df}{dx}\right]_{x_i}$

So at every iteration, we move close to $x^*$ having started from some random point.

If $|x_{k+1} - x_k|$ is really low, stop. $x^* = x_k$

If $f$ is a vector func$^n$:

$$x_{i+1} = x_i - r\left[\nabla_x f(x)\right]_{x_i}$$




$\leftarrow \left(\frac{df}{dx}\right)_{x_0} > \left(\frac{df}{dx}\right)_{x_1} > \cdots$

jumps would get smaller as we move towards $x^*$.

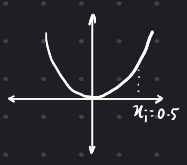
# Learning rate/ step-size

In the update eq$^n$, $x_i = x_{i-1} - r\left[\frac{df}{dx}\right]_{x_{i-1}}$ $r$ is the learning rate.

If we keep $r$ constant:

Say $r = 1$, $f(x) = x^2$, $x_i = 0.5$

$x_{i+1} = x_i - r\left[\frac{df}{dx}\right]_{x_i}$




$x_{i+1} = 0.5 - 1(2 \times 0.5) = -0.5$

Now we got the point $x = -0.5$

We simply jumped over to the other side.

$x_{i+2} = -0.5 - 1(-1) = 0.5$

Again the next point is $0.5$.

So at this learning rate, we'll just keep oscillating betⁿ $0.5$ & $-0.5$.

We'll never converge to $x^*$.

Remedy for oscillation: change $r$ with each iteration.

1 way: reduce $r$ a little each time.

$r$ becomes a funcⁿ of iteration #. $r = h(i)$ s.t. $r \propto \frac{1}{i}$

# Gradient descent for linear regression

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - w^T x_i)^2$$

... for simplicity, we'll ignore regularization term and intercept term.

$$L(w) = \sum_{i=1}^{n} (y_i - w^T x_i)^2$$

$$\nabla_w L = \sum_{i=1}^{n} \left[ 2(y_i - w^T x_i)(-x_i) \right]$$

① Pick a random $w_0 = \langle \cdots \rangle$

② $w_1 = w_0 - \eta \sum_{i=1}^{n} (-2x_i)(y_i - w_0^T x_i)$

③ $w_2 = w_1 - \eta \sum_{i=1}^{n} (-2x_i)(y_i - w_1^T x_i)$

$\vdots$

$w_k = w_{k-1} - \eta \sum_{i=1}^{n} (-2x_i)(y_i - w_{k-1}^T x_i)$

Problem: If $n$ is large (say a million), computing $\sum_{i=1}^{n} (-2x_i)(y_i - w_{j-1}^T x_i)$ for every iter$^n$ is really expensive.

Workaround: Stochastic Gradient Descent

---

# Stochastic Gradient Descent (SGD)

$$w_{j+1} = w_j - \eta \underbrace{\sum_{i=1}^{n} (-2x_i)(y_i - w_j^T x_i)}_{\text{this takes a lot of time.}}$$

Instead of calculating the sum over $i=1$ to $n$, we'll just do it for some $k$ points. $k \leq n$.

$$w_{j+1} = w_j - \eta \cdot \sum_{i=1}^{k} (-2x_i)(y_i - w^T x_i)$$

The solution with SGD would be similar to that with GD.

So, say we needed 100 iter$^n$s of GD to converge to $w^*$, for $n = 1$ million

SGD would probably take > 100 iter$^n$s. Say 500, for $k = 1000$

1000 for $k = 10$

$k \rightarrow$ the batch size, is different for every iter$^n$ of SGD.

When $1 < k \leq n$, SGD is called batch SGD.

But # of computations is quite lesser.

# Constrained Optimization

Optimizaⁿ problems may have constraints.

General constrained optimizaⁿ:

$$\max_x \underbrace{f(x)}_{\text{obj. funcⁿ}} \quad \text{s.t.} \quad \underbrace{g(x) = c}_{\text{equality constraint}}, \quad \underbrace{h(x) \geq d}_{\text{inequality constraint}}.$$

We'll modify this using Lagrangian multipliers.

$$\mathcal{L}(x, \lambda, \mu) = f(x) - \lambda(g(x) - c) - \mu(d - h(x))$$

$\mu, \lambda \geq 0$ are Lagrangian multipliers.

We solve $\frac{\partial \mathcal{L}}{\partial x} = 0$, $\frac{\partial \mathcal{L}}{\partial \lambda} = 0$, $\frac{\partial \mathcal{L}}{\partial \mu} = 0$ to get $\tilde{x}$. $\tilde{x}$ will be the $\max_x f(x)$, i.e. $x^*$, given the constraints.

---

Consider the PCA formulaⁿ: $\max_u \underbrace{\frac{1}{n}\sum_{i=1}^{n}(u^T x_i)^2}_{\text{objective funcⁿ}}$ s.t. $\underbrace{u^T u = 1}_{\text{constraint}}$

can be transformed to

$\max_u u^T S u$ s.t. $u^T u = 1$ and $S = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^T$

↳ covariance matrix of $X$ (provided $X$ is centred and scaled)

$$L(u, \lambda) = u^T S u - \lambda(u^T u - 1)$$

$$\frac{\partial L}{\partial u} = 0$$

$$Su - \lambda u = 0$$

$$Su = \lambda u$$

$\therefore u$ is the eigen vector, and $\lambda$ is the eigen value of $S$.

---

Logistic regression:

$$w^* = \underset{w}{\text{argmin}}(\text{logistic loss}) + \underbrace{\lambda w^T w}_{\text{similar to Lagrange multiplier}}$$

$$w^* = \underset{w}{\text{argmin}}(\text{logistic loss}) \quad \text{s.t.} \quad w^T w = 1.$$

} constraint optimization problem.

$$L = \text{logistic loss} - \lambda(1 - w^T w)$$

$$= \text{logistic loss} - \lambda + \lambda w^T w$$

Regularization can be thought of as imposing an equality constraint.

# L1 regularization creates sparsity

$L_1$ regularizaⁿ creates sparsity in $w = \langle w_1, \dots, w_d \rangle$

i.e. a lot of $w_i$'s will be zeroes.

let's compare $L_1$ and $L_2$ reg$^n$:

$L_2$

$$\min_w \text{loss} + \lambda \|w\|_2^2$$

$L_1$

$$\min_w \text{loss} + \lambda \|w\|_1$$

let's ignore loss (since it is the same for both) and $\lambda$ (since we'll choose a fixed value)

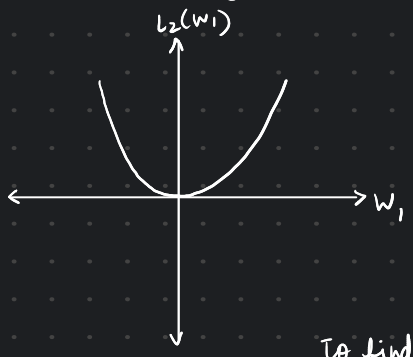
$$\min_{w_1, w_2 \dots w_d} (w_1^2 + w_2^2 + \dots + w_d^2)$$

$$\min_{w_1 \dots w_d} (|w_1| + |w_2| + \dots + |w_d|)$$

effectively

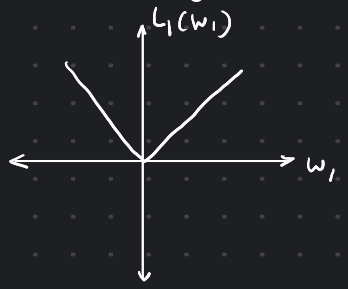
$$\min_{w_1} w_1^2$$

↳ say $L_2(w_1)$



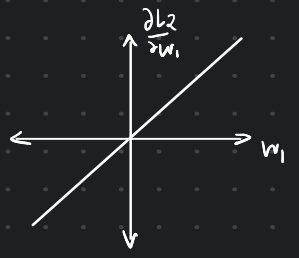

$$\min_{w_1} |w_1|$$

↳ say $L_1(w_1)$

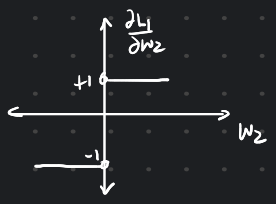



To find minima, we'll look at their derivatives.

$$\frac{\partial L_2}{\partial w_1} = 2w_1$$




$$\frac{\partial L_1}{\partial w_1} = \begin{cases} +1 & ; w_1 > 0 \\ -1 & ; w_1 < 0 \end{cases}$$




updates: (in gradient descent)

$(w_1)_0, (w_1)_1, (w_1)_2, \dots, (w_1)_k$

$$(w_1)_{j+1} = (w_1)_j - r \left[\frac{\partial L_2}{\partial w_1}\right]_{w_j}$$

$$(w_1)_{j+1} = (w_1)_j - r(2w_{1j})$$

$$(w_1)_{j+1} = (w_1)_j - r \left[\frac{\partial L_1}{\partial w_1}\right]_{w_j}$$

assume $w_{1j} > 0$

$$(w_1)_{j+1} = (w_1)_j - r[1]$$

only difference

In both, $\min_w(w) = 0$ ↓ ↓

Say at $i^{th}$ iteraⁿ,

$(w_1)_j = 0.05$

so $r(2w_{1j})$ becomes really small as well. Here $r(1)$ is constant.

If $r$ becomes really small, say 0.01

$2w_{1j}\, r = 2 \times 0.05 \times 0.01 = 0.001$ $\qquad r = 0.01$

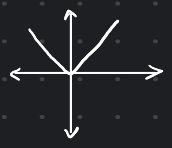
so $w_{j+1} = 0.5 - 0.001$

so as we go closer to $w^*$, derivative decreases




$w_{1j} = 0.5 - 0.01$

here it is constant



So $L_2$ regularizer does not change the value of $w_i$ from one iteration to the other.

$L_1$ regularizer continues to decrease the value to $w_i \to 0$.

(gradient is constant)

So at the end of k iterations, the chances that $w_i = 0$ are more.

So with $L_1$ regularization, more of the weights corresponding to useless/less imp. features will become zero.

---